

An Evaluation of Neural Network Architectures for Predicting Hydro Unit Commitment Decisions

Daria Barjaktarevic¹, Jiehong Kong², Zhirong Yang^{1,3}, Christian Øyn Naversen²

¹Department of Computer Science, Norwegian University of Science and Technology, Trondheim, Norway

²Department of Renewable Energy, SINTEF Energy Research, Trondheim, Norway

³Jinhua Institute of Zhejiang University, Jinhua, China

Abstract—This paper evaluates a range of neural network (NN) architectures for predicting short-term unit commitment (UC) decisions in a hydro-dominated power system. A dataset is generated using an operational hydro scheduling tool for a real Norwegian watercourse. We compare simple, convolutional, recurrent, and hybrid NN models for multilabel UC prediction over 24-h and 168-h scheduling horizons. All models achieve high predictive performance, with F1-scores above 97.35% and clear improvements over the relaxed Mixed-Integer Programming (No_MIP) baseline. Convolutional architectures exhibit the most consistent results across both horizons. When embedded into the scheduling pipeline, machine-learning-predicted UC decisions reduce median runtime by up to 64.4% while maintaining objective values close to those obtained with an exact MIP solution.

Index Terms—Hydro Unit Commitment, Machine Learning, Mixed Integer Programming, Neural Network.

I. INTRODUCTION

In recent decades, numerous optimization models have been developed to improve the utilization of limited resources in power systems. With rapid advances in computing hardware, algorithms, and data availability, research on Machine Learning (ML) for power system optimization has grown markedly [1, 2]. Unlike conventional optimization methods, which repeatedly solve similar problems without accumulating experience from previous instances, ML-based approaches can extract patterns from historical data and past operational decisions. Numerical studies have demonstrated that such learning-augmented techniques can substantially boost optimization performance [2].

The Unit Commitment (UC) problem is one of the most fundamental and computationally intensive tasks in power system operations [3]. It is typically formulated as a Mixed-Integer Programming (MIP) problem in which binary variables determine the on/off status of generating units and continuous variables govern their dispatch levels. The binary nature of the problem is essential for representing realistic technical

constraints, such as minimum production levels, but it greatly increases computational cost.

In short-term hydro-dominant UC problems, the modeling complexity is even greater. Short-term hydro scheduling requires a detailed representation of head effects, hydraulic losses, nonlinear production functions, and complex watercourse topologies. This leads to high-dimensional input data, fine time resolution, and thousands of technical and hydraulic constraints [4]. Temporal coupling arises because water stored in reservoirs can be used later, creating intertemporal trade-offs between immediate generation and future flexibility. Spatial coupling emerges in cascaded hydro systems, where releases from upstream plants directly influence downstream production opportunities. Together, these spatiotemporal dependencies considerably expand the combinatorial structure of the feasible region.

Although state-of-the-art commercial solvers such as CPLEX [5] continue to improve, solving hydro-dominant MIP formulations to tight optimality remains time-consuming. The challenge is especially pronounced in daily bidding and dispatching operations, where the problem must be solved repeatedly within limited time windows. Moreover, recent changes in the Nordic electricity market design, i.e., the transition of the market time unit from one hour to 15 minutes, further increase the computational burden [6].

Recently, several ML methodologies have been presented to effectively extract information from previously solved cases to improve the computational performance of UC problems. A comprehensive review in [7] highlights ML applications for predicting UC decisions, either as a complete or a partial decision schedule, or as a warm start for the optimization solver. The study concludes that ML-based predictions can yield substantial benefits, with time savings of up to 85% when all binary variables are predicted directly.

Among the ML models used for UC prediction, logistic regression, decision trees, random forests, and Neural Networks (NNs) are the most explored. NNs, in particular, have demonstrated strong predictive performance. Prior studies have

This paper is supported by the Research Council of Norway through the Knowledge-building Project for Industry “Deep integration between machine learning approaches and renewable energy optimization” (InterOpt) under Grant award no. 352879.

primarily focused on feedforward architecture [8-13], with a few cases considering graph-based NNs [14] or recurrent NNs such as the Long Short-Term Memory (LSTM) network [15] or Gated Recurrent Unit [16, 17]. However, the literature still lacks a systematic comparison of NN architectures to determine which are most suitable for UC prediction tasks. Furthermore, most studies rely on standard IEEE bus systems [18], which do not capture the hydraulic and operational dynamics of hydro-dominated systems. This limits their practical relevance in markets such as Norway, where hydropower plays a central role [19].

This paper addresses these gaps by investigating the use of ML to support UC decisions in short-term hydro scheduling. Instead of relying on simplified assumptions, we use the Short-term Hybrid Optimization Program (SHOP), an operational scheduling tool developed by SINTEF Energy Research [20], to generate a training dataset for a real Norwegian watercourse. The detailed physical modeling in SHOP ensures that the resulting ML tasks capture the operational intricacies and hydraulic dependencies in real hydro scheduling environments. Both the watercourse dataset [21] and the complete training dataset [22] are publicly available to support reproducibility.

We evaluate a broad set of NN architectures, including convolutional and recurrent models, to predict UC decisions from economic signals and hydraulic states. Model performance is quantified using accuracy, precision, recall, and the F1 score. ML-predicted UC decisions are subsequently applied in SHOP to assess their operational impact. We then compare ML-predicted schedules with exact, relaxed, and mixed MIP settings in terms of objective value and computational time. Our findings indicate that ML models can capture much of the UC decision structure while significantly reducing computational effort.

The paper is organized as follows. Section II explains the short-term hydro UC problem. Section III details the dataset generation pipeline, including case design, feature structure, and label extraction from SHOP solutions. Section IV describes the different NN architectures. Section V presents the findings, including metrics, runtimes, and impact on objective values, and Section VI concludes the paper.

II. SHORT-TERM HYDRO UNIT COMMITMENT PROBLEM

Short-term hydro scheduling determines how water is used over the next hours and days to balance economic value, hydraulic constraints, and operational requirements. Scheduling horizons typically span from one day to one week, with time resolutions from 15 minutes to one hour. Because unit on/off decisions interact with nonlinear hydraulic relationships and reservoir dynamics, the resulting optimization problem is large-scale, mixed-integer, and nonconvex. A detailed description of the underlying modeling principles and solution strategies can be found in [23].

As an operational scheduling tool widely used by Nordic hydropower producers, SHOP employs a two-stage solution strategy. In the UC mode, the problem is formulated as a Mixed-Integer Linear Program (MILP) whose primary objective is to determine the on/off schedules for all units. Reservoir states are updated iteratively across the watercourse, typically converging within three to five iterations. In the

subsequent Unit Load Dispatch (ULD) mode, the binary UC decisions are fixed, and the problem is solved as a Linear Program (LP) to compute production levels for all committed units. A full description of SHOP's formulations and algorithms is provided in [20].

In practice, to reduce computational effort, binary UC variables are often enforced only for the first 24-48 hours of the planning horizon and relaxed thereafter. Relaxation allows UC variables to take continuous values in $[0, 1]$, which may result in production levels falling below technical minimums. Current operational practice handles this by applying a static threshold to determine whether a unit should be treated as "off." In this paper, we explore an alternative approach in which ML is used to determine UC status after MIP relaxation.

To examine ML-based UC prediction under realistic operational conditions, we use the Tokke-Vinje watercourse in southern Norway. Once the country's largest hydropower development [24], the system comprises 17 reservoirs, 9 power plants, and 14 generating units with a total installed capacity of 1,000 MW and an annual production of 4.9 TWh [25]. Physical characteristics and historical hydrologic records are sourced from the Norwegian Water Resources and Energy Directorate (NVE)'s open Sildre [26] and DETD [27] datasets and then refined by SINTEF Energy Research. The watercourse is in price zone NO2, and hourly historical prices from ENTSO-E [28] are used to represent market conditions. The complete watercourse dataset used in this paper can be downloaded from [21].

All optimization tasks in this study are solved using SHOP v17.15.1 with CPLEX v20.1.0 as the solver. Following the optimization phase, SHOP performs a high-resolution hydraulic simulation at a one-second (s) time step. This simulation re-evaluates the production schedules using the original nonlinear hydropower production function and physical water routing [29], rather than the piecewise linear approximations employed in the ULD mode during optimization. Consequently, the simulation-based objective value represents a more accurate assessment of the schedule's physical performance, i.e., the amount of water required to achieve the given production levels. This metric is therefore used to evaluate the operational quality of exact, relaxed, mixed, and ML-predicted UC solutions. The overall workflow is illustrated in Figure 1.

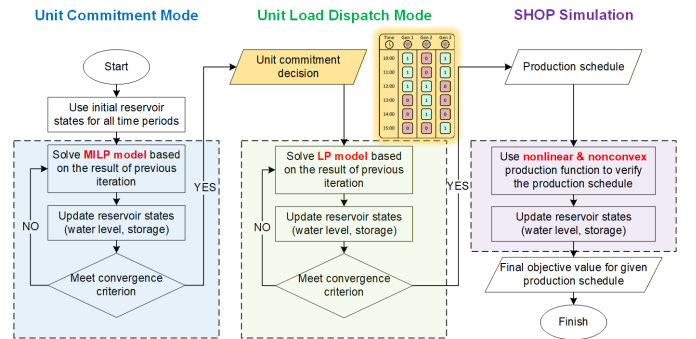


Figure 1. Iterative optimization and simulation process in SHOP.

III. DATASET GENERATION

This section describes how the ML dataset is constructed, how truth labels are selected, and how the final ML-ready

structure is formed. Short-term hydro scheduling in a competitive electricity market relies on four time-varying inputs: reservoir initial states (water level or storage), inflows, end water values, and market prices. The collection and computation of these time series are documented in [30], and the open dataset in [25] provides 10 years of hourly input data (2015-01-01 to 2024-12-31) for all reservoirs and the NO2 price zone. Based on these inputs, two scheduling horizons, 24-hours (h) and 168 h, are tested. SHOP is executed in a daily rolling-horizon manner, yielding 3,653 cases for the 24-h horizon and 3,647 cases for the 168-h horizon.

Each case is solved under three operational settings in the UC mode:

1) *All_MIP*: solves five iterations as MILP, using a zero MIP gap for 24-h cases and an absolute gap of 1,000 € or a relative gap of 0.001% for 168-h cases, whichever is met first.

2) *No_MIP*: relaxes all binary UC variables and performs five LP iterations. If the optimized production falls below 50% of the minimum operating limit, the unit is set to “off” (0); otherwise, it is committed as “on” (1).

3) *Mixed_MIP*: applies a strategy with three relaxed LP iterations followed by two MILP iterations. The relaxed LP iterations are used to quickly update reservoir states at each timestep, providing more accurate hydraulic conditions before the final MILP iterations determine the UC decisions.

ULD mode always performs five LP iterations using the UC decisions obtained from UC mode. All computations are solved in parallel using 16 workers, one solver thread per case with a 900-s time limit, executed on a workstation equipped with an Intel® Core™ i9-13900K CPU (24 cores) and 64 GB RAM. The dataset generation time for each horizon is summarized in TABLE I. It demonstrates that the relaxed settings substantially reduce computation time, with *No_MIP* requiring only around 20% of the runtime of the *All_MIP* setting.

TABLE I. DATASET GENERATION TIME AND SIZE

Horizon	Generation time (s)			Dataset size		
	<i>All_MIP</i>	<i>No_MIP</i>	<i>Mixed_MIP</i>	No. of Case	Feature columns	Label columns
24-h	2,892	528	1,204	3,653	1,162	336
168-h	44,663	7,276	18,945	3,647	7,930	2,352

After SHOP optimization, each candidate schedule undergoes the one-second-resolution hydraulic simulation, yielding physically consistent objective values. The UC schedule that produces the highest simulation objective value is selected as the truth label, ensuring that the label corresponds to the operationally best UC plan under the full nonlinear physics. In case of ties, decisions are selected in the priority order *All_MIP* → *Mixed_MIP* → *No_MIP*.

The final ML dataset merges all input data, CPLEX solver outputs from the *No_MIP* setting, and truth labels into a single feature matrix. For a watercourse with r reservoirs, l catchment rivers, and k generating units over n scheduling periods, features include: n prices, $n \times r$ reservoir inflows, $n \times l$ catchment inflows, r initial reservoir states, r end water values, $n \times k$ production outputs, $n \times k$ UC outputs. Labels consist of the $n \times k$ binary UC decisions selected as truth. For the Tokke–Vinje system ($r = 17, l = 1, k = 14$), the resulting dataset dimensions are listed in TABLE I. The datasets, available at [22], exhibit a highly unbalanced structure, with many more

columns (features + labels) than rows (cases), due to the limited length of historical data relative to the model’s spatiotemporal dimensionality.

IV. NEURAL NETWORK ARCHITECTURES

An NN consists of layers of interconnected nodes, or neurons, through which data are transformed. The input layer receives the features, hidden layers apply learned transformations, and the output layer produces predictions. Each node contains learnable weights applied to data from the previous layer, and the transformed data are passed on to the next layer. Between layers, an activation function is applied. The prediction and the true label are evaluated using a loss function, which is then used to update the weights via backpropagation. The input layer itself contains no learnable weights and serves only as a placeholder for the input data.

An NN can predict continuous, binary, or integer variables depending on the activation function used at the output layer. For multilabel binary classification, a sigmoid activation produces probabilities that are thresholded to obtain binary values. The number of layers and layer sizes of an NN are usually determined through hyperparameter tuning.

A. Fully Connected Networks

A Multilayer Perceptron (MLP) [31] is an NN with at least one hidden layer, and all layers are fully connected. MLPs are scalable and can model complex relationships. In addition, DLinear was proposed in [32] as an approach for time-series forecasting and has shown strong performance compared to other state-of-the-art models. It decomposes data into trend and seasonal components using a moving-average kernel. Although originally developed for forecasting, we use DLinear here for sequence-to-sequence prediction.

B. Convolutional Neural Networks

A Convolutional Neural Network (CNN) [33] uses convolution operations to progressively extract information from data. CNNs were initially used for image classification, where early layers learn simple features such as edges and corners, which are subsequently composed into more complex patterns. CNNs are also effective for modeling structured data with nonlinear relationships.

In a convolutional layer, a kernel of chosen dimension slides across the data and computes the convolution between the data and the kernel weights. The kernel weights are learnable, meaning CNNs learn “locally”. For time-series applications, one-dimensional convolutional kernels are applied across timesteps rather than across features/input channels. Unlike MLPs, CNNs are not fully connected.

An ordinary CNN requires many layers to obtain a large receptive field, defined as the number of input nodes that influence a single output node. For sequences with long-term dependencies, this can be restrictive. Dilated convolutions (Figure 2b) address this problem by expanding the receptive field without increasing the number of parameters. One CNN architecture that exploits this idea is the Circular Dilated CNN (CDIL-CNN) [34], combining dilation with circular padding.

For time-series prediction, it is often desirable that predictions at time t depend only on information up to time t . This motivation leads to causal convolutions. Instead of

convolving symmetrically around the previous node, kernels convolve only with previous timesteps (Figure 2c). The Temporal Convolutional Neural Network (TCN) [35] integrates both dilated and causal convolutions to handle time-series data with complex long-term dependencies.

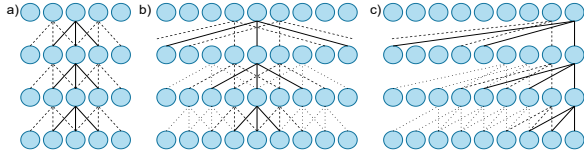


Figure 2. CNNs with kernel size 3: a) Ordinary CNN. b) Dilated convolutions. c) Causal dilated convolutions.

C. Recurrent Neural Networks

The Recurrent Neural Network (RNN) [36] was designed to model sequential data such as time series, speech, or text. A standard RNN consists of a recurrent unit containing a hidden state that processes the input sequentially. The recurrent connection allows the model to “remember” what has happened previously in the sequence.

While RNNs are effective for short sequences, they suffer from vanishing gradients, which limit their ability to model long-term dependencies. Sequential processing also makes training and prediction unparallelizable. The LSTM network [37] addresses the vanishing-gradient problem and therefore enables modeling of long-term dependencies. Essentially, an LSTM cell maintains both a hidden state and a long-term memory, updated using learnable gates that regulate information flow. These gates allow the model to identify which long-term dependencies are important (Figure 3b).

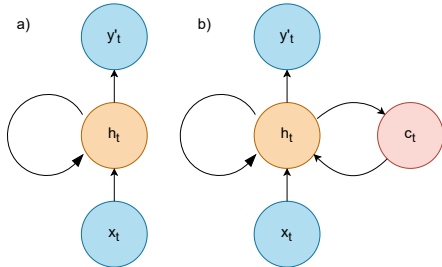


Figure 3. Two different types of recurrence: a) Ordinary recurrent unit, where the previous hidden state is an input to the node along with the next step in the sequence. b) LSTM recurrent unit, where the long-term memory is also input to the unit and is updated after the new hidden state is calculated.

D. Other Architectures

In our evaluation, we included two composite NNs: LSTM-CNN and CNN-LSTM. These consist of an LSTM followed by a CNN, and a CNN followed by an LSTM, respectively. Such a hybrid architecture functions as an automatic feature extractor, with the first part learning key features and the second combining them into predictions.

V. RESULTS AND ANALYSIS

All models are trained for both the 24-h and 168-h scheduling horizons. We randomly allocate 80% of the cases for training and use the remaining 20% for evaluation. All experiments are conducted on an Apple M4 Pro chip (24GB RAM). The models are implemented in PyTorch v2.9.1 [38] using Python v3.12.7, and hyperparameter tuning is done with

Optuna v2.10.1 [39]. Optimal models are trained using the Metal Performance Shaders backend.

Requiring the entire UC schedule to match exactly would be overly strict and would not provide meaningful insight into operational quality. We therefore treat each UC decision at each timestep as an individual label and evaluate model performance using accuracy, precision, recall, and an F1 score with a micro average. After ML-predicted UC decisions are generated, they are fed back into the SHOP workflow, i.e., running ULD mode followed by simulation, as illustrated in Figure 1, to obtain objective values and calculation times as operational performance measures.

A. Performance of Neural Networks Compared to No_MIP

Figure 4 and Figure 5 show that all NNs achieve strong performance, with F1 scores of at least 98.23% for the 24-h horizon and 97.48% for the 168-h horizon. This strong performance may seem surprising given the data's scarcity, but we expect our models to perform at least as well as the baseline *No_MIP*. Compared to 98.21% and 97.35% for *No_MIP*, we see a solid but reasonable increase in performance from the baseline. Note that *No_MIP* has a recall close to 100% in both cases, a level few NNs match, but its precision is comparatively low. This is due to *No_MIP*'s heuristic threshold for commitment: any unit with production above 50% of its minimum limit is set to “on,” leading to systematic over-commitment and increased false positives.

The performance differences between NN architectures are relatively small. CNN-based models exhibit slightly better overall results, suggesting that convolutional layers effectively capture the temporal and structural dependencies inherent to hydro scheduling. DLinear performs surprisingly well for the 24-h case but fails to maintain this performance for the 168-h horizon, indicating that the simple network cannot handle longer sequences. MLP also performs well given its simplicity. In contrast, recurrent models show the weakest overall performance, except for the ordinary LSTM.

MLP	0.9849	0.9774	0.9916	0.9844
DLinear	0.9867	0.9808	0.9919	0.9863
CNN	0.9858	0.9803	0.9905	0.9853
CDIL-CNN	0.9861	0.9803	0.9912	0.9857
TCN	0.9859	0.9804	0.9905	0.9854
RNN	0.9828	0.9763	0.9884	0.9823
LSTM	0.9837	0.9769	0.9895	0.9832
CNN-LSTM	0.9832	0.9789	0.9864	0.9827
LSTM-CNN	0.9845	0.9797	0.9884	0.9840
No MIP	0.9824	0.9656	0.9991	0.9821
	Accuracy	Precision	Recall	F1 Micro

Figure 4. Classification performance metrics for models trained on the 24-h scheduling horizon.

MLP	0.9764	0.9675	0.9851	0.9762
DLinear	0.9770	0.9710	0.9826	0.9768
CNN	0.9777	0.9733	0.9815	0.9774
CDIL-CNN	0.9781	0.9743	0.9813	0.9778
TCN	0.9773	0.9730	0.9811	0.9770
RNN	0.9762	0.9682	0.9838	0.9760
LSTM	0.9764	0.9720	0.9801	0.9760
CNN-LSTM	0.9752	0.9718	0.9779	0.9748
LSTM-CNN	0.9758	0.9730	0.9778	0.9754
No MIP	0.9734	0.9529	0.9951	0.9735
	Accuracy	Precision	Recall	F1 Micro

Figure 5. Classification performance metrics for models trained on the 168-h scheduling horizon.

B. Training, Prediction, and Total Pipeline Times

TABLE II. shows that NN training times are modest due to the limited cases available. The simplest models, MLP and DLinear, are significantly faster than the others, while RNN is noticeably slower. Prediction times follow the same pattern. In general, larger models (i.e. many learnable parameters) require longer training and prediction times, with RNN being a notable exception. Note that the training times are the time required to train one instance of the model, excluding hyperparameter tuning.

However, training and prediction times alone do not represent the runtime in a full operational workflow. An ML-enhanced SHOP run consists of three steps: 1) obtaining the relaxed solution in UC mode, 2) predicting UC decisions using the NN, and 3) running the model with the ML-predicted UC decisions in ULD mode. Figure 6 shows violin plots of total runtime across different SHOP operational settings, evaluated for the 20% of cases reserved for prediction. CDIL-CNN, similar to other NN models, exhibits impressive speed-ups, reducing the median runtime by 56.1% and 64.4% compared to *All_MIP* for the 24-h and 168-h horizons, respectively.

TABLE II. LEARNABLE PARAMETERS, TRAINING AND PREDICTION TIME

Model	24 hours			168 hours		
	Para- meters	Train (s)	Predict (ms)	Para- meters	Train (s)	Predict (ms)
MLP	14,094	24.28	0.206 ± 0.024	60,734	126.38	0.267 ± 0.028
DLinear	2,544	26.97	0.365 ± 0.040	58,128	108.87	0.478 ± 0.098
CNN	55,836	56.36	0.708 ± 0.046	388,092	201.00	1.135 ± 0.234
CDIL-CNN	66,508	56.90	0.891 ± 0.095	262,236	198.43	1.577 ± 0.262
TCN	59,556	72.76	1.124 ± 0.341	257,900	194.76	2.221 ± 0.327
RNN	4,078	104.72	3.948 ± 0.222	4,078	594.37	19.632 ± 1.017
LSTM	14,926	60.11	1.038 ± 0.338	14,926	227.20	4.392 ± 1.097
CNN-LSTM	49,470	54.33	0.902 ± 0.084	36,894	224.49	4.471 ± 1.141
LSTM-CNN	25,766	62.19	0.918 ± 0.043	112,430	221.55	5.032 ± 1.731

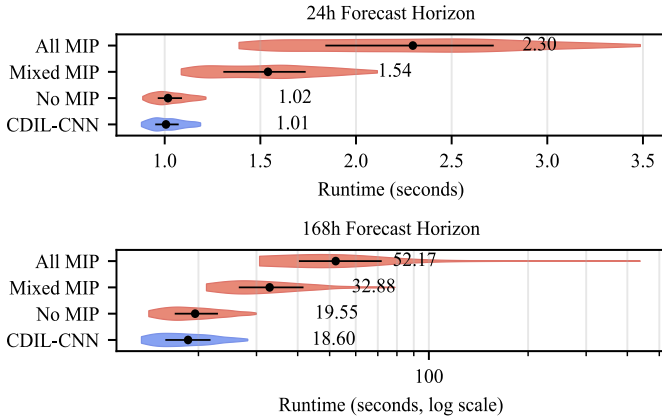


Figure 6. Violin plots of total runtimes for different SHOP operational settings and the CDIL-CNN, showing the central 80% of data. Median values are shown.

C. Impact of ML-predicted UC Decisions on Objective Values

The goal of ML-predicted UC decisions is to improve, or at least preserve, operational performance compared to *All_MIP*. Figure 7 shows violin plots of the objective value differences between the *All_MIP* benchmark and alternative SHOP operational settings, i.e., *No_MIP*, *Mixed_MIP*, and ML-enhanced runs. The results show that *All_MIP* generally achieves the best median performance, although the differences are not consistently large across all models. Importantly, ML-enhanced runs exhibit performance comparable to *No_MIP*, with some models even showing notable improvements.

However, models with the best classification metrics (Figure 4 and Figure 5) do not always achieve the largest objective-value improvements. This highlights the importance of evaluating ML predictions within the full optimization loop, rather than relying solely on classification metrics.

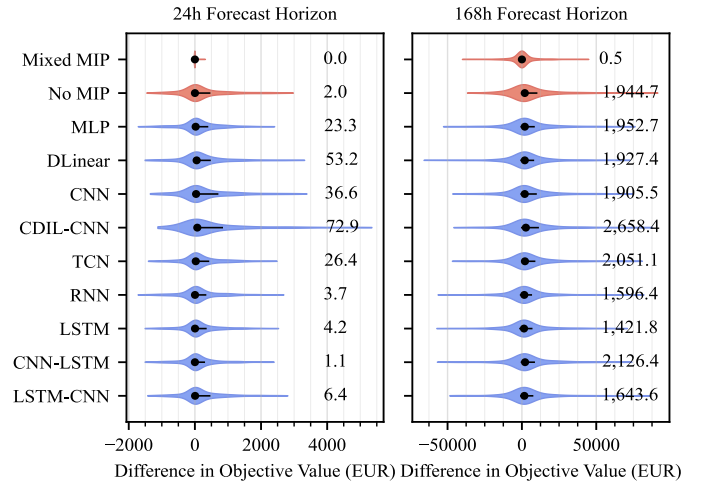


Figure 7. Violin plots of objective-value differences (*All_MIP* – model), showing the central 80% of the data. Median values are indicated. Larger values represent greater deviations from the *All_MIP* objective value.

VI. CONCLUSIONS

This study evaluated a range of NN architectures to predict hydro UC decisions using data generated by a detailed operational scheduling tool. The results show that all tested architectures achieved high predictive performance, with F1 scores exceeding 97.35% for both short and long scheduling horizons and consistently outperformed the *No_MIP* baseline. Convolutional architectures demonstrated the strongest and most stable performance overall, suggesting that their ability to capture local temporal patterns and structured dependencies is particularly well suited to hydro scheduling tasks.

When integrated into the scheduling pipeline, ML-predicted UC decisions significantly reduced the total runtime while maintaining objective values close to those of an exact MIP solution. However, the observed variability in economic outcomes indicates that conventional classification metrics alone do not fully capture the operational quality of predicted schedules. These findings demonstrate that NNs can effectively approximate hydro UC decisions and enable meaningful computational savings, while also emphasizing the importance of evaluating ML models within the full optimization context.

REFERENCES

- [1] J. Bernardes, Jr. *et al.*, "Hydropower operation optimization using machine learning: a systematic review," *AI*, vol. 3, no. 1, pp. 78–99, 2022, doi: 10.3390/ai3010006.
- [2] G. C. Ruan, H. W. Zhong, G. L. Zhang, Y. L. He, X. Wang, and T. J. Pu, "Review of learning-assisted power system optimization," *CSEE Journal of Power and Energy Systems*, vol. 7, no. 2, pp. 221–231, Mar 2021, doi: 10.17775/cseejpes.2020.03070.
- [3] Y. Yang and L. Wu, "Machine learning approaches to the unit commitment problem: Current trends, emerging challenges, and new strategies," *The Electricity Journal*, vol. 34, no. 1, 2021, doi: <https://doi.org/10.1016/j.tej.2020.106889>.
- [4] J. Kong, H. I. Skjelbred, and O. B. Fosso, "An overview on formulations and optimization methods for the unit-based short-term hydro scheduling problem," *Electric Power Systems Research*, vol. 178, 2020, doi: 10.1016/j.epsr.2019.106027, in press.
- [5] "IBM ILOG CPLEX Optimizer." IBM. <https://www.ibm.com/products/ilog-cplex-optimization-studio/cplex-optimizer> (accessed 3 March, 2026).
- [6] "Transition to 15-minute Market Time Unit (MTU)," Nord Pool. <https://www.nordpoolgroup.com/en/trading/transition-to-15-minute-market-time-unit-mtu/> (accessed 29 January, 2026).
- [7] J. Milla and J. Pérez-Díaz, "The Use of Artificial Intelligence for the Unit Commitment Problem: State of the Art," in *ELECTRIMACS 2024*, vol. 1275, pp. 713–725, doi: 10.1007/978-3-031-73921-7_54.
- [8] M. Shekeew and B. Venkatesh, "Learning-Assisted Variables Reduction Method for Large-Scale MILP Unit Commitment," *IEEE Open Access Journal of Power and Energy*, vol. 10, pp. 245–258, 2023, doi: 10.1109/OAJPE.2023.3247989.
- [9] A. Ramesh and X. Li, "Feasibility Layer Aided Machine Learning Approach for Day-Ahead Operations," *IEEE Trans. Power Syst.*, vol. 39, no. 1, pp. 1582–1593, 2024, doi: 10.1109/TPWRS.2023.3266192.
- [10] N. Sugishita, A. Grothey, and K. McKinnon, "Use of Machine Learning Models to Warmstart Column Generation for Unit Commitment," *INFORMS Journal on Computing*, vol. 36, no. 4, pp. 1129–1146, 2024–01-29 2024, doi: 10.1287/ijoc.2022.0140.
- [11] M. Li, W. Wei, Y. Chen, M. Ge, and J. Catalao, "Learning the Optimal Strategy of Power System Operation With Varying Renewable Generations," *IEEE Transactions on Sustainable Energy*, vol. 12, no. 4, pp. 2293–2305, 2021–10-01 2021, doi: 10.1109/TSTE.2021.3088951.
- [12] Z. Lin *et al.*, "Accelerating transmission-constrained unit commitment via a data-driven learning framework," *Frontiers in Energy Research*, vol. 10, 2023–01-11 2023, Art no. 1012781, doi: 10.3389/fenrg.2022.1012781.
- [13] Z. Weng, X. Tang, P. Li, X. Bai, and L. Zheng, "A fast ES-based method for solving SCUC problem," *Electr. Power Syst. Res.*, vol. 227, 2023–11-06 2024, Art no. 109978, doi: 10.1016/j.epsr.2023.109978.
- [14] S. Schmitt, I. Harjunkoski, M. Giuntoli, J. Poland, and X. Feng, "Fast Solution of Unit Commitment Using Machine Learning Approaches," in *2022 IEEE 7th International Energy Conference (ENERGYCON)*, pp. 1–6, doi: 10.1109/ENERGYCON53164.2022.9830191.
- [15] A. Ramesh and X. Li, "Spatio-Temporal Deep Learning-Assisted Reduced Security-Constrained Unit Commitment," *IEEE Trans. Power Syst.*, vol. 39, no. 2, pp. 4735–4746, 2024–03-01 2024, doi: 10.1109/TPWRS.2023.3313430.
- [16] N. Yang *et al.*, "Deep learning-based SCUC decision-making: An intelligent data-driven approach with self-learning capabilities," *IET Generation Transmission & Distribution*, vol. 16, no. 4, pp. 629–640, 2021–10-16 2022, doi: 10.1049/gtd2.12315.
- [17] J. Hao, N. Yang, D. Ye, Y. He, C. Yang, and X. Zhang, "The Research of a Data-Driven Intelligent SCUC Decision-making Approach based on Gated Recurrent Unit and Seq2Seq Technique," in *2021 International Conference on Cyber-Physical Social Intelligence (ICCSI)*, 2021–01-01 2021, doi: 10.1109/ICCSI5130.2021.9736250.
- [18] S. Pineda and J. M. Morales, "Is learning for the unit commitment problem a low-hanging fruit?," *Electr. Power Syst. Res.*, vol. 207, Jun 2022, Art no. 107851, doi: 10.1016/j.epsr.2022.107851.
- [19] A. Helseth *et al.*, "Hydropower scheduling toolchains: Comparing experiences in Brazil, Norway, and USA and implications for synergistic research," *J. Water Resour. Plan. Manage.-ASCE*, vol. 149, no. 7, Jul 2023, Art no. 04023030, doi: 10.1061/jwrmd5.wreng-5911.
- [20] H. I. Skjelbred, J. Kong, and O. B. Fosso, "Dynamic incorporation of nonlinearity into MILP formulation for short-term hydro scheduling," *Int. J. Electr. Power Energy Syst.*, vol. 116, Mar 2020, Art no. 105530, doi: 10.1016/j.ijepes.2019.105530.
- [21] J. Kong and C. Ø. Naversen. *Tokke-Vinje Watercourse Open Dataset v1_2025_11*. [Online]. Available: <https://data.sintef.no/feature/fe-cc13261c-8bf8-4a2b-a76b-9281980a7414>
- [22] J. Kong, D. Barjaktarevic, and C. Ø. Naversen. *KSP InterOpt Project Open Dataset SHOP 17_15_1 Tokke_3 history*. [Online]. Available: <https://data.sintef.no/feature/fe-a3e8b5ef-6128-4910-87b5-ad46c39a5f01>
- [23] J. H. Kong, H. I. Skjelbred, and O. B. Fosso, "An overview on formulations and optimization methods for the unit-based short-term hydro scheduling problem," *Electr. Power Syst. Res.*, vol. 178, Jan 2020, Art no. 106027, doi: 10.1016/j.epsr.2019.106027.
- [24] "Norway Tokke-Vinje Water System." Statkraft Energi AS. https://www.statkraft.com/globalassets/0/-com/1-about-statkraft/where-we-operate/norway/norwaytokke-vinje_eng.pdf (accessed 3 March, 2026).
- [25] J. Kong, S. Tiwari, C. Ø. Naversen, T. Å. Reite, H. I. Skjelbred, and K. A. Tyse, "Norwegian Open Dataset: Tokke-Vinje Watercourse," 2025.
- [26] "Public portal for hydrological time series from Norwegian Water Resources and Energy Directorate." NVE. <https://sildre.nve.no/> (accessed 30 January, 2026).
- [27] "Model dataset of Norwegian and Nordic hydropower." NVE. <https://www.nve.no/energi/energisystem/vannkraft/modell-av-norsk-og-nordisk-vannkraft/> (accessed 3 March, 2026).
- [28] ENTSO-E. "European Network of Transmission System Operators for Electricity (ENTSO-E)." <https://www.entsoe.eu/> (accessed 5 March, 2026).
- [29] H. I. Skjelbred and J. Kong, "Operational hydropower simulation in cascaded river systems for intraday re-planning," presented at the 20th Power Systems Computation Conference (PSCC), Dublin, IRELAND, 11–15 June, 2018.
- [30] T. Å. Reite, J. Kong, and C. Ø. Naversen, "Synthetic data generation for machine learning-accelerated hydropower scheduling: A scalable approach to overcome historical data limitations," working paper, submitted to EEM 2026, 2026.
- [31] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986, doi: 10.1038/323533a0.
- [32] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are Transformers Effective for Time Series Forecasting?," in *2023 Proc. of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 9, pp. 11121–11128.
- [33] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," in *1998 Proc. Of the IEEE*, vol. 86, no. 11, pp. 2278–2324, doi: 10.1109/5.726791.
- [34] L. Cheng, R. Khalitov, T. Yu, J. Zhang, and Z. Yang, "Classification of long sequential data using circular dilated convolutional neural networks," *Neurocomputing*, vol. 518, pp. 50–59, 2022–11-08 2023, doi: 10.1016/j.neucom.2022.10.054.
- [35] C. Lea, R. Vidal, A. Reiter, and G. Hager, "Temporal Convolutional Networks: A Unified Approach to Action Segmentation," in *Computer Vision – ECCV 2016 Workshops*, vol. 9915, pp. 47–54, doi: 10.1007/978-3-319-49409-8_7.
- [36] J. L. Elman, "Finding Structure in Time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990, doi: 10.1207/s15516709cog1402_1.
- [37] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
- [38] "PyTorch." PyTorch Foundation. <https://pytorch.org/> (accessed 3 March, 2026).
- [39] "Optuna – A Hyperparameter Optimization Framework [optuna.org]." Preferred Networks, Inc. <https://optuna.org/> (accessed 3 March, 2026).